



60 min = La Hora de Oro

IES Castelar · 2026

GOLDEN HOUR

Planificación de rutas seguras
con cobertura hospitalaria garantizada

Documento de Especificación de Requisitos

Proyecto Final de Ciclo · BioErgon Lab

Angular

Spring Boot

PostgreSQL + PostGIS

Leaflet.js

¿Qué vamos a ver hoy?

La presentación está dividida en 4 bloques

01

El problema y la idea

Por qué existe este proyecto

02

Los Requisitos (SRS)

Qué tendría que hacer la aplicación

03

Cómo se montaría

Arquitectura y funcionamiento planificado

04

Planificación por Sprints

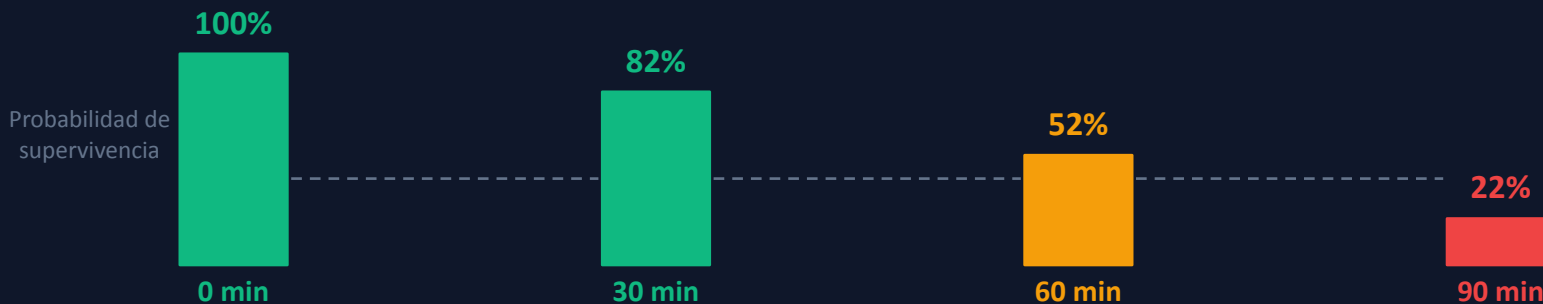
Cómo organizaríamos el desarrollo

¿Qué es la «Hora de Oro»?

El concepto médico que da nombre al proyecto

En medicina de urgencias existe una regla que llevan décadas usando los servicios de emergencias:

Si una persona con una lesión grave recibe atención médica especializada en los primeros 60 minutos, sus posibilidades de sobrevivir se multiplican. A eso se le llama la «Golden Hour» o Hora de Oro.



El Problema en Extremadura

¿Por qué nace Golden Hour aquí y no en otro sitio?

>20%

del territorio español está a más de 60 min de un hospital

41.634 km²

es Extremadura — una de las comunidades más grandes y menos densas de España

0 apps

existen ahora mismo que avisen de esto al planificar una ruta por carretera



¿Qué haría Golden Hour?

Explicado en una frase antes de entrar en los requisitos

La idea central: una app que calcula rutas buscando que nunca estés a más de 60 minutos de un hospital.

GPS normal

- ✓ Calcula la ruta más rápida
- ✓ Avisa del tráfico
- ✓ Recalcula si te pierdes
- ✗ NO sabe si hay hospital cerca
- ✗ NO avisa de zonas peligrosas

VS

Golden Hour

- ✓ Calcularía la ruta más rápida
- ✓ Localizaría hospitales en el mapa
- ✓ Analizaría CADA tramo de la ruta
- ✓ Avisaría si te alejas de hospitales
- ✓ Sugeriría rutas alternativas seguras



El Semáforo de Seguridad

La idea más visual del proyecto — y la más importante



02

Los Requisitos del SRS

Usuarios · Alcance · Requisitos Funcionales · Requisitos No Funcionales

¿Quién usaría la app?

Tres tipos de usuario con permisos distintos

No todos los usuarios necesitarían lo mismo. Por eso se plantean tres roles:

Invitado	✓ Ver el mapa con hospitales ✓ Calcular cualquier ruta ✓ Ver el semáforo de la ruta ✗ Guardar rutas ✗ Tener perfil
Registrado	✓ Todo lo del invitado ✓ Guardar rutas favoritas ✓ Perfil de salud ✗ Panel de admin ✗ Editar hospitales
Admin	✓ Todo lo anterior ✓ CRUD de hospitales ✓ Gestionar usuarios ✗ (Acceso total)



Login

Registro

Alcance del Proyecto

Qué Sí entraría en la v1.0 y qué se dejaría para después

Dejar claro el alcance desde el principio evita malentendidos. Una versión 1.0 tiene que ser realista:

Sí entraría en la v1.0

- ✓ Calcular rutas entre dos puntos
- ✓ Ver hospitales en el mapa
- ✓ Semáforo de seguridad por tramos
- ✓ Login y registro de usuarios
- ✓ Guardar rutas favoritas
- ✓ Filtrar hospitales por especialidad
- ✓ Panel de administración básico

NO entraría (queda para v2.0)

- ✗ Navegación GPS giro a giro
- ✗ Tráfico en tiempo real
- ✗ Pedir cita médica desde la app
- ✗ App móvil nativa (Android/iOS)
- ✗ Botón SOS de emergencia
- ✗ Tiempos de espera en urgencias

Requisitos Funcionales (1/2)

¿Qué tendría que hacer el sistema? — RF-01 a RF-05

Un RF describe una acción concreta que la app tendría que ser capaz de realizar:

RF-01	¿Hay apps similares?	Analizar el mercado y detectar qué falta	Alta
RF-02	¿Puedo calcular mi ruta?	Introducir origen/destino y ver la ruta en el mapa	Alta
RF-03	¿Dónde están los hospitales?	Mostrar todos los hospitales como marcadores en el mapa	Alta
RF-04	¿Cuánto tardaría al hospital?	Calcular tiempo al hospital más cercano por cada tramo	CRITICA
RF-05	¿Como sé si es seguro?	Colorear tramos: verde <30 min, naranja 30-60, rojo >60	Alta

Requisitos Funcionales (2/2)

RF-06 a RF-10

RF-06	¿Puedo guardar mis rutas?	Guardar rutas favoritas con nombre en el perfil	Media
RF-07	¿Puedo ver info del hospital?	Popup con nombre, telefono y especialidades al hacer clic	Media
RF-08	¿Puedo filtrar hospitales?	Filtrar por especialidad medica (pediatria, trauma...)	Baja
RF-09	¿Quién gestiona los datos?	Panel de admin: añadir, editar y borrar hospitales (CRUD)	Media
RF-10	¿Y si hay zonas rojas?	Sugerir una ruta alternativa mas segura automaticamente	Alta

Requisitos No Funcionales

No dicen QUE haría la app — dicen COMO tendría que ser

Una app que hace todo pero tarda 10 segundos por petición no sirve de nada. Los RNF definen la calidad:

Usabilidad

RNF-01

Primera ruta calculada en menos de 3 clics. Sin manual.

Rendimiento

RNF-02

Cálculo completo en menos de 3 segundos.

Open Source

RNF-03

Todo gratis: OSM, OSRM, PostGIS, Spring Boot, Angular.

Precision

RNF-04

Tiempos por carretera real (OSRM), no en línea recta.

Seguridad

RNF-05

Contraseñas con BCrypt.
Sesiones con JWT firmado.

Responsive

RNF-06

Funcionaria en móvil (320px) y escritorio (1920px).

Mantenibilidad

RNF-07

API documentada con Swagger en /swagger-ui.html.

03

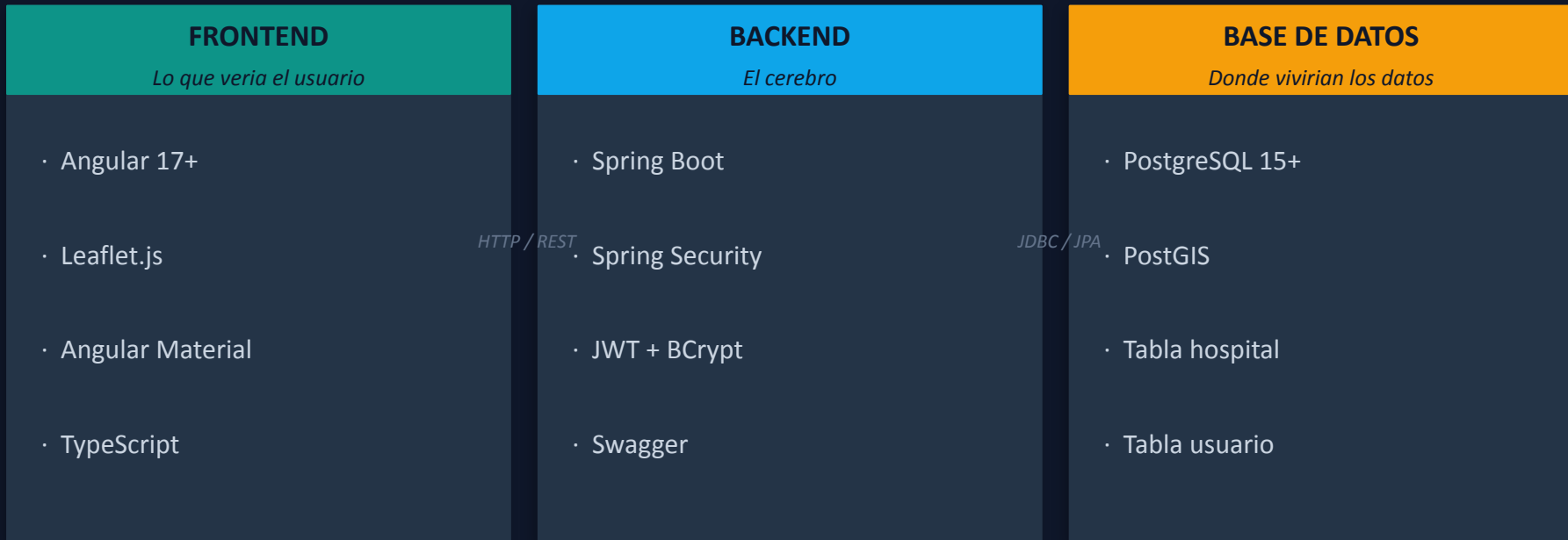
Como se montaria

Mi propuesta de arquitectura, tecnologias, modelo de datos y seguridad

La Arquitectura: 3 Capas bien separadas

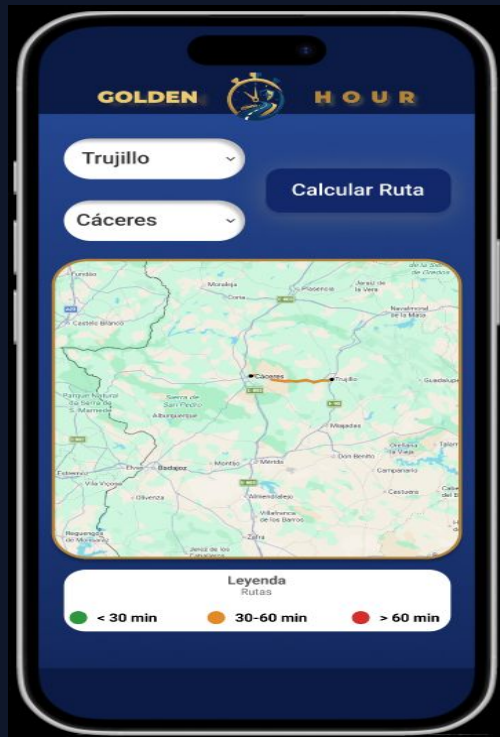
Como he planificado organizar las partes de la aplicacion

He planificado dividir la app en 3 partes separadas que se comunican entre si. A esto se le llama arquitectura de 3 capas:



El Frontend Propuesto: Angular + Leaflet

Lo que el usuario veria e interactuaria con ello



Mapa con semaforo de ruta

Angular 17+

Organiza la app en componentes TypeScript reutilizables. Todo tiene su lugar.

Leaflet.js

Pintaria el mapa interactivo y los tramos coloreados del semaforo.

Angular Material

Componentes de UI listos: botones, formularios, navegacion... Ahorra tiempo.

RxJS

Gestiona las llamadas al servidor de forma asincrona con Observables.

El Backend Propuesto: Spring Boot

El servidor que recibiría las peticiones y aplicaría la lógica

Endpoints principales (API REST)

GET	/api/hospitales	Devuelve todos los hospitales
POST	/api/validar-ruta	Calcula el semáforo
POST	/auth/login	Login del usuario
POST	/auth/registro	Registro de usuario
POST	/api/rutas/guardar	Guarda una ruta
GET	/api/rutas/mias	Rutas del usuario

Spring Security + JWT

Protegería los endpoints. Solo usuarios con token válido podrían acceder.

Spring Data JPA

Conectaría objetos Java con tablas PostgreSQL. Sin escribir SQL a mano.

Swagger / OpenAPI

Generaría automáticamente la documentación en /swagger-ui.html.

PostGIS: el truco que lo hace posible

Por que no podria funcionar igual con una BD normal

PostgreSQL es la base de datos que usaria. PostGIS es una extension que le aniaade superpoderes geograficos. Sin ella, el proyecto no existiria tal como lo he diseñado.

Sin PostGIS (distancia en linea recta)

«El hospital esta a 15 km...» — Pero hay una sierra de por medio.

El tiempo real puede ser 70 minutos, no 20.

Con PostGIS + OSRM (tiempo real por carretera)

«70 minutos por la N-630» — ESO si tiene utilidad real.

```
-- Encuentra el hospital mas cercano:
SELECT
  nombre,
  ST_Distance(
    ubicacion::geography,
    ST_SetSRID(
      ST_MakePoint(-6.2, 38.9),
      4326
    )::geography
  ) AS metros
FROM hospital
ORDER BY metros ASC
LIMIT 1;
```

Esto devuelve el hospital mas cercano y la distancia — solo SQL, cero Java.

Servicios externos: todo gratis

Sin pagar ni un euro a Google Maps

Una decision importante fue no usar Google Maps API porque cuesta dinero y tiene limites de uso. En su lugar usaria estas tres alternativas gratuitas y open source:

1

MAPA

OpenStreetMap

Proporciona las imagenes del mapa (tiles). Es la Wikipedia de los mapas — mantenida por voluntarios de todo el mundo. Completamente gratis.

2

RUTAS

OSRM

Open Source Routing Machine. Calcula la ruta mas rapida por carretera entre dos coordenadas GPS y devuelve la polyline (lista de puntos GPS).

3

GEOCODING

Nominatim

Convierte texto en coordenadas GPS. Cuando escribes «Badajoz», Nominatim devuelve latitud 38.88 y longitud -6.97. Sin esto no habria buscador.

OpenStreetMap tiene mas de 10 millones de contribuidores. Es la Wikipedia de los mapas.

El Modelo de Datos

Las 3 tablas que tendria la base de datos

La base de datos tendria 3 tablas. Cada una guardaria un tipo distinto de informacion:

USUARIO	HOSPITAL	RUTA GUARDADA
<code>id</code> Identificador unico	<code>id</code> Identificador unico	<code>id</code> Identificador unico
<code>email</code> Sirve de nombre de usuario	<code>nombre</code> Nombre del centro	<code>id_usuario</code> A que usuario pertenece
<code>password_hash</code> BCrypt — nunca en texto claro	<code>ubicacion</code> 1:N Tipo PostGIS — el mas importante —	<code>nombre_ruta</code> Ej: «Trabajo a casa»
<code>nombre</code> Visible en la interfaz	<code>direccion</code> Direccion postal	<code>coord_origen</code> Punto de partida
<code>rol</code> Define los permisos	<code>telefono_urgencias</code> Para el popup del mapa	<code>coord_destino</code> Punto de llegada
<code>fecha_registro</code> Se pone automaticamente	<code>servicios</code> Lista de especialidades	<code>es_segura</code> True si no hay tramos rojos

¿Que pasa por dentro al calcular una ruta?

Paso a paso: desde que el usuario pulsa «Calcular» hasta que ve los colores



El Login: JWT para identificar al usuario

¿Como sabria el servidor quien eres en cada peticion?

HTTP no tiene memoria — cada peticion que llega al servidor es nueva, como si no conociera. JWT soluciona eso con un token firmado que el usuario lleva en cada peticion.

1

Mandas email
y contraseña

2

BCrypt compara
con el hash guardado

3

Si es correcto, genera
un TOKEN JWT firmado

4

Angular guardaria el
token
y lo enviaria en cada
peticion

5

El backend verificaria
la firma del token

6

Si es valida,
devuelve los datos



04

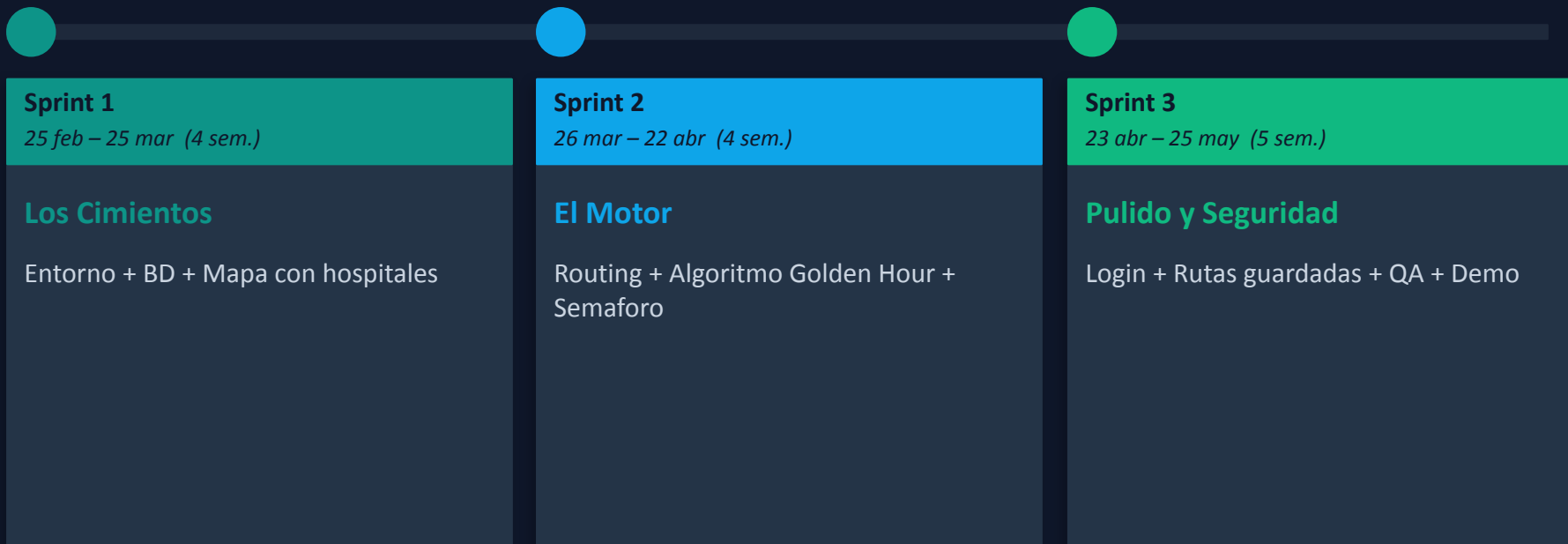
Planificacion por Sprints

25 febrero – 25 mayo 2026 · 3 sprints · ~13 semanas

Los 3 Sprints: Vision General

25 febrero – 25 mayo 2026

La idea es ir de menos a mas: primero que funcione algo basico, luego añadir el algoritmo principal, y al final dejarlo todo bonito y seguro.



Sprint 1 — 25 feb al 25 mar

Los cimientos: que el mapa muestre hospitales

Objetivo: tener el entorno montado, la base de datos con hospitales reales de Extremadura y el mapa pintandolos. Nada mas, pero que funcione bien.

Sprint 1 / 3 · 4 semanas

25 feb – 7 mar - Analisis y Setup

- Investigar apps similares (RF-01)
- Instalar Angular CLI, Spring Boot, Docker
- Levantar PostgreSQL con PostGIS en Docker
- Crear proyectos base en el repositorio Git

8 mar – 18 mar - Datos de Hospitales

- Crear la tabla hospital en PostgreSQL
- Cargar datos reales de hospitales extremenos
- Crear endpoint GET /api/hospitales
- Probar con Postman que devuelve JSON

19 mar – 25 mar - Mapa con Hospitales

- Integrar Leaflet en Angular (ngx-leaflet)
- Consumir la API y pintar marcadores
- Popup de info del hospital al hacer clic
- Revision y ajustes finales

Sprint 2 — 26 mar al 22 abr

El motor: el algoritmo de la Hora de Oro

Objetivo: que la app calcule rutas y las coloree segun el tiempo al hospital mas cercano. Esta parte es el nucleo del proyecto, la mas divertida y la mas dificil.

Sprint 2 / 3 · 4 semanas

26 mar – 5 abr - El Routing

- Formulario origen/destino en Angular
- Geocodificacion con Nominatim
- Conexion con OSRM para la polyline (RF-02)
- Pintar la ruta sin colores todavia

6 abr – 13 abr - El Algoritmo

- Endpoint POST /api/validar-ruta
- Segmentar la ruta en puntos intermedios
- Consulta PostGIS para cada punto (RF-04)
- Retornar tramos con nivel de riesgo

14 abr – 22 abr - El Semaforo

- Pintar tramos coloreados en Leaflet (RF-05)
- Panel de filtros por especialidad (RF-08)
- Sugerir ruta alternativa si hay rojos (RF-10)
- Testing y correcciones

Entreeable: ruta con semaforizacion verde / naranjia / roja completamente funcional.

Sprint 3 — 23 abr al 25 may

La recta final: login, demo y entregar

Objetivo: que la app sea completa, con login, rutas guardadas y todo probado. Las ultimas 2 semanas son solo para pulir detalles y preparar la demo.

Sprint 3 / 3 · 5 semanas — recta final!

23 abr – 3 may - Login y Registro

- Spring Security + JWT en el backend
- Hash BCrypt para las contraseñas
- Pantallas de Login y Registro en Angular
- Proteger endpoints con @PreAuthorize

4 may – 14 may - Rutas Guardadas

- Endpoint POST /api/rutas/guardar
- Tabla ruta_guardada en la BD
- Vista «Mis Rutas» en Angular
- Eliminar rutas guardadas (RF-06)

15 may – 25 may - QA + Demo

- Tests de rendimiento (< 3 seg)
- Responsive en movil y tablet
- Documentacion Swagger
- Preparar demo y esta presentacion

Entregable final: app completa, segura, documentada y funcionando en demo.



GOLDEN HOUR

2 DAW · IES Castelar · 2026